

- 算法竞赛常用库与操作指南
 -  目录
 - 1. C++ STL 常用容器
 - 1.1 vector (动态数组)
 - 1.2 deque (双端队列)
 - 1.3 queue (队列)
 - 1.3 stack (栈)
 - 1.4 unordered_map (哈希表)
 - 1.5 map (有序映射)
 - 1.6 set / unordered_set (集合)
 - 1.7 priority_queue (优先队列/堆)
 - 1.8 pair 和 tuple
 - 2. 字符串处理
 - 2.1 stringstream (字符串流)
 - 2.2 getline (读取整行)
 - 2.3 字符串基本操作
 - 2.4 复杂字符串解析
 - 3. 输入输出操作
 - 3.1 C++ 标准输入输出
 - 3.2 C 标准输入输出
 - 3.3 文件输入输出
 - C++ 文件操作
 - C 文件操作
 - 3.5 多行输入模式
 - 5. 数据结构构建模板
 - 5.1 链表
 - 5.2 二叉树
 - 7. 实用技巧与注意事项
 - 7.1 常用头文件汇总
 - 7.2 常见陷阱
 - 7.3 性能优化技巧

算法竞赛常用库与操作指南

本文档基于 homework 文件夹下的实际作业代码整理，涵盖 C、C++、Java 三种语言的常用操作。

- 1. C++ STL 常用容器
- 2. 字符串处理
- 3. 输入输出操作
- 4. 常用算法库
- 5. 数据结构构建模板
- 6. Java 常用操作
- 7. 实用技巧与注意事项

1. C++ STL 常用容器

1.1 vector（动态数组）

参考文件: [homework/2026_3_20/Solution1.cpp](#),
[homework/2026_4_4/maxSlidingWindow.cpp](#)

```
#include <vector>

// 声明与初始化
vector<int> nums;           // 空向量
vector<int> nums(10, 0);    // 大小为10, 初始值为0
vector<long long> prefix(n + 1, 0); // 前缀和数组常用写法

// 基本操作
nums.push_back(x);         // 尾部添加元素
nums.pop_back();           // 删除尾部元素
nums.size();               // 获取大小
nums.empty();              // 判断是否为空
nums[i];                   // 访问第i个元素
nums.clear();              // 清空向量

// 遍历
for (int i = 0; i < nums.size(); i++) {
    cout << nums[i] << " ";
}

// 范围for循环 (C++11)
for (auto num : nums) {
    cout << num << " ";
}
```

```
// 二维向量
vector<vector<int>> matrix(n, vector<int>(m, 0));
```

1.2 deque（双端队列）

参考文件: [homework/2026_4_4/maxSlidingWindow.cpp](#),
[homework/2026_4_4/substring.cpp](#)

```
#include <deque>

// 声明
deque<int> dq;

// 基本操作
dq.push_back(x);      // 尾部添加
dq.push_front(x);     // 头部添加
dq.pop_back();        // 尾部删除
dq.pop_front();       // 头部删除
dq.front();           // 访问队首元素
dq.back();            // 访问队尾元素
dq.empty();           // 判断是否为空
dq.size();            // 获取大小

// 滑动窗口典型用法
while (!dq.empty() && dq.front() <= right - windowSize) {
    dq.pop_front();    // 移除超出窗口的元素
}

while (!dq.empty() && nums[dq.back()] < nums[right]) {
    dq.pop_back();     // 维护单调性
}

dq.push_back(right);  // 存入下标而非值
```

1.3 queue（队列）

参考文件: [homework/2026_4_24/maxway.cpp](#)

```
#include <queue>

// 声明
queue<TreeNode*> q;

// 基本操作
q.push(node);        // 入队
q.pop();              // 出队
```

```

q.front();           // 访问队首
q.empty();           // 判断是否为空
q.size();             // 获取大小

// BFS层序遍历典型用法
queue<TreeNode*> q;
q.push(root);
while (!q.empty()) {
    TreeNode* cur = q.front();
    q.pop();
    // 处理当前节点
    if (cur->left) q.push(cur->left);
    if (cur->right) q.push(cur->right);
}

```

1.3 stack（栈）

参考文件: [homework/2026_3_26/cal.c](#)（C语言手动实现）

```

#include <stack>

// 声明
stack<int> stk;

// 基本操作
stk.push(x);           // 入栈
stk.pop();             // 出栈
stk.top();             // 访问栈顶
stk.empty();           // 判断是否为空
stk.size();            // 获取大小

// 表达式求值典型用法
while (!opStk.empty() && getPriority(opStk.top()) >= getPriority(currentOp)) {
    char op = opStk.top(); opStk.pop();
    int b = numStk.top(); numStk.pop();
    int a = numStk.top(); numStk.pop();
    numStk.push(calc(a, b, op));
}

```

1.4 unordered_map（哈希表）

参考文件: [homework/2026_3_20/Solution1.cpp](#),
[homework/2026_4_10/groupAnagrams.cpp](#)

```

#include <unordered_map>

```

```

// 声明
unordered_map<int, int> map; // key-value都是int
unordered_map<string, vector<string>> groupMap; // key是string, value是vector

// 基本操作
map[key] = value; // 插入或更新
map.count(key); // 判断key是否存在（返回0或1）
map.find(key) == map.end(); // 判断key是否存在
map.erase(key); // 删除键值对
map.size(); // 获取大小
map.clear(); // 清空

// Two Sum 典型用法
unordered_map<int, int> map;
for (int i = 0; i < nums.size(); i++) {
    if (map.count(target - nums[i])) {
        return {map[target - nums[i]], i};
    }
    map[nums[i]] = i;
}

// 分组典型用法
unordered_map<string, vector<string>> map;
for (auto str : strs) {
    string key = str;
    sort(key.begin(), key.end());
    map[key].push_back(str);
}

```

1.5 map（有序映射）

```

#include <map>

// 声明
map<int, string> orderedMap;

// 基本操作（与unordered_map类似，但按键排序）
orderedMap[key] = value;
orderedMap.find(key);
orderedMap.lower_bound(key); // 第一个>=key的迭代器
orderedMap.upper_bound(key); // 第一个>key的迭代器

// 遍历（自动按键排序）
for (auto& [key, value] : orderedMap) {
    cout << key << ": " << value << endl;
}

```

1.6 set / unordered_set（集合）

```

#include <set>
#include <unordered_set>

// 声明
set<int> orderedSet;           // 有序集合
unordered_set<int> hashSet;    // 哈希集合

// 基本操作
hashSet.insert(x);            // 插入
hashSet.erase(x);            // 删除
hashSet.count(x);             // 判断是否存在
hashSet.find(x) != hashSet.end(); // 查找
hashSet.size();               // 大小
hashSet.clear();              // 清空

// 去重典型用法
unordered_set<int> seen;
for (int num : nums) {
    if (seen.count(num)) {
        // 重复元素
    }
    seen.insert(num);
}

```

1.7 priority_queue（优先队列/堆）

```

#include <queue>

// 最大堆（默认）
priority_queue<int> maxHeap;

// 最小堆
priority_queue<int, vector<int>, greater<int>> minHeap;

// 自定义结构体比较
struct Node {
    int val;
    int idx;
};

struct Compare {
    bool operator()(const Node& a, const Node& b) {
        return a.val > b.val; // 最小堆
    }
};

priority_queue<Node, vector<Node>, Compare> pq;

// 基本操作
pq.push(node); // 入堆
pq.pop();      // 出堆
pq.top();      // 访问堆顶

```

```
pq.empty();           // 判断是否为空
pq.size();            // 大小
```

1.8 pair 和 tuple

```
#include <utility>
#include <tuple>

// pair
pair<int, int> p = make_pair(1, 2);
p.first;    // 访问第一个元素
p.second;   // 访问第二个元素

// C++11 简化写法
auto p = make_pair(1, 2);
pair<int, int> p{1, 2};

// tuple
tuple<int, string, double> t = make_tuple(1, "hello", 3.14);
get<0>(t);   // 访问第一个元素
get<1>(t);   // 访问第二个元素
```

2. 字符串处理

2.1 stringstream（字符串流）

参考文件: [homework/2026_3_20/Solution1.cpp](#),
[homework/2026_4_4/maxSlidingWindow.cpp](#)

```
#include <sstream>
#include <string>

// 从字符串中读取整数序列
string line = "1 2 3 4 5";
stringstream ss(line);
vector<int> nums;
int num;
while (ss >> num) {
    nums.push_back(num);
}

// 从字符串中读取混合类型
string data = "100 hello 3.14";
```

```
stringstream ss(data);
int a;
string b;
double c;
ss >> a >> b >> c;

// 重置stringstream
ss.str(""); // 清空内容
ss.clear(); // 清除错误标志
ss.str(newString); // 设置新内容
```

2.2 getline（读取整行）

参考文件: [homework/2026_3_20/Solution1.cpp](#),
[homework/2026_4_4/maxSlidingWindow.cpp](#)

```
#include <string>
#include <iostream>

// 从cin读取整行
string line;
getline(cin, line);

// 从文件读取整行
ifstream fin("input.txt");
string line;
while (getline(fin, line)) {
    // 处理每一行
}

// 指定分隔符
string token;
getline(ss, token, ','); // 以逗号为分隔符
```

2.3 字符串基本操作

```
#include <string>
#include <algorithm>

string s = "hello";

// 基本操作
s.length(); // 长度
s.empty(); // 是否为空
s.substr(start, len); // 子串
s.find(sub); // 查找子串位置
```



```

s.append(str);          // 追加
s += str;               // 追加

// 字符串转换
int num = stoi(s);      // string -> int
long num = stol(s);     // string -> long
double num = stod(s);   // string -> double
string s = to_string(num); // 数字 -> string

// 排序
sort(s.begin(), s.end()); // 字母异位词判断常用

// 分割字符串（手动实现）
vector<string> split(string s, char delimiter) {
    vector<string> tokens;
    string token;
    istringstream iss(s);
    while (getline(iss, token, delimiter)) {
        tokens.push_back(token);
    }
    return tokens;
}

```

2.4 复杂字符串解析

参考文件: [homework/2026_4_24/maxway.cpp](#)

```

// 解析 root = [1,2,3,null,null,15,7]
vector<string> parseInput(string line) {
    // 去掉空格
    string content;
    for (char c : line) {
        if (c != ' ') content += c;
    }

    // 提取方括号内容
    int l = content.find('[');
    int r = content.find(']');
    content = content.substr(l + 1, r - l - 1);

    // 按逗号分割
    vector<string> vals;
    string cur;
    for (char c : content) {
        if (c == ',') {
            vals.push_back(cur);
            cur.clear();
        } else {
            cur += c;
        }
    }
    if (!cur.empty()) vals.push_back(cur);
}

```

```
    return vals;
}
```

3. 输入输出操作

3.1 C++ 标准输入输出

参考文件: [homework/2026_4_4/canteen.cpp](#),
[homework/2026_4_10/groupAnagrams.cpp](#)

```
#include <iostream>

// 基本输入
int n;
cin >> n;

int a, b;
cin >> a >> b;

string s;
cin >> s; // 读取到空白为止

// 读取一行
string line;
getline(cin, line);

// 注意: cin >> 后使用 getline 需要处理换行符
cin >> n;
cin.ignore(); // 忽略换行符
getline(cin, line);

// 基本输出
cout << "Hello" << endl;
cout << a << " " << b << endl;
printf("%d %d\n", a, b); // 也可以使用printf

// 加速IO (竞赛常用)
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

3.2 C 标准输入输出

参考文件: [homework/2026_3_20/S7.c](#), [homework/2026_3_26/cal.c](#)

```
#include <stdio.h>

// 基本输入
int n;
scanf("%d", &n);

int a, b;
scanf("%d %d", &a, &b);

char str[100];
scanf("%s", str);

// 读取整行
char line[1000];
fgets(line, 1000, stdin);

// 循环读取直到EOF
int temp;
while (scanf("%d", &temp) != EOF) {
    // 处理每个数字
}

// 基本输出
printf("%d\n", n);
printf("%d %d\n", a, b);
printf("%s\n", str);

// 格式化输出
printf("%.2f\n", 3.14159); // 保留两位小数
printf("%05d\n", 42);      // 补零输出: 00042
```

3.3 文件输入输出

C++ 文件操作

参考文件: [homework/2026_3_20/Solution1.cpp](#)

```
#include <fstream>
#include <iostream>
#include <string>

// 方法1: ifstream/ofstream
ifstream fin("input.txt");
if (!fin.is_open()) {
    cerr << "文件打开失败" << endl;
    return 1;
}
```

```

string line;
while (getline(fin, line)) {
    // 处理每一行
}
fin.close();

// 写入文件
ofstream fout("output.txt");
fout << "Hello World" << endl;
fout.close();

// 方法2: freopen (重定向标准输入输出)
freopen("input.txt", "r", stdin); // 从文件读取
freopen("output.txt", "w", stdout); // 输出到文件

// 之后正常使用 cin/cout
int n;
cin >> n;
cout << n << endl;

```

C 文件操作

参考文件: [homework/2026_4_17/LCA.c](#)

```

#include <stdio.h>

// 打开文件
FILE *fp = fopen("input.txt", "r");
if (fp == NULL) {
    printf("文件打开失败\n");
    return 1;
}

// 读取数据
int n;
fscanf(fp, "%d", &n);

int u, v;
fscanf(fp, "%d%d", &u, &v);

char str[100];
fscanf(fp, "%s", str);

// 关闭文件
fclose(fp);

// 写入文件
FILE *fout = fopen("output.txt", "w");
fprintf(fout, "%d\n", result);
fclose(fout);

```

3.5 多行输入模式

参考文件: [homework/2026_3_20/Solution1.cpp](#)

```
// 模式1: 成对读取
ifstream fin("input.txt");
string numsLine, targetLine;
while (getline(fin, numsLine)) {
    if (!getline(fin, targetLine)) break;

    // 处理 numsLine 和 targetLine
    stringstream ss(numsLine);
    vector<int> nums;
    int x;
    while (ss >> x) {
        nums.push_back(x);
    }
    int target = stoi(targetLine);

    // 计算并输出结果
}
fin.close();

// 模式2: 先读数量, 再读数据
int n;
cin >> n;
vector<int> nums(n);
for (int i = 0; i < n; i++) {
    cin >> nums[i];
}
```

5. 数据结构构建模板

5.1 链表

参考文件: [homework/2026_3_20/S7.c](#)

```
// C++ 链表节点
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};
```

```

// C 链表节点
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// 创建链表（从输入）
Node* head = (Node*)malloc(sizeof(Node));
Node* tail = head;

if (scanf("%d", &(head->data))) {
    head->next = NULL;
    int temp;
    while (scanf("%d", &temp) != EOF) {
        Node* t = (Node*)malloc(sizeof(Node));
        t->data = temp;
        tail->next = t;
        tail = t;
        tail->next = NULL;
    }
}

// 遍历链表
Node* p = head;
while (p != NULL) {
    printf("%d ", p->data);
    p = p->next;
}

```

5.2 二叉树

参考文件: [homework/2026_4_24/maxway.cpp](#)

```

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// 从层序数组建树
TreeNode* buildTree(vector<string>& vals) {
    if (vals.empty() || vals[0] == "null") return nullptr;

    TreeNode* root = new TreeNode(stoi(vals[0]));
    queue<TreeNode*> q;
    q.push(root);

    int i = 1;
    while (!q.empty() && i < vals.size()) {
        TreeNode* cur = q.front();
        q.pop();

```

```

// 左子节点
if (i < vals.size() && vals[i] != "null") {
    cur->left = new TreeNode(stoi(vals[i]));
    q.push(cur->left);
}
i++;

// 右子节点
if (i < vals.size() && vals[i] != "null") {
    cur->right = new TreeNode(stoi(vals[i]));
    q.push(cur->right);
}
i++;
}

return root;
}

// DFS遍历
void dfs(TreeNode* root) {
    if (!root) return;
    // 处理当前节点
    dfs(root->left);
    dfs(root->right);
}

// BFS遍历
void bfs(TreeNode* root) {
    if (!root) return;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        TreeNode* cur = q.front();
        q.pop();
        // 处理当前节点
        if (cur->left) q.push(cur->left);
        if (cur->right) q.push(cur->right);
    }
}

```

@所有人 各位同学：555焊接实验的时间段已安排完毕，请同学们自行选择合适的时间，填写共享文档，并按时参加实验。如有任何问题，请联系实验室张典老师，QQ：3391753636。

7. 实用技巧与注意事项

7.1 常用头文件汇总

```

// C++ 万能头文件（竞赛可用，工程不推荐）
#include <bits/stdc++.h>

// 或者按需包含
#include <iostream>      // cin, cout
#include <fstream>       // 文件操作
#include <sstream>       // stringstream
#include <string>        // string
#include <vector>        // vector
#include <deque>         // deque
#include <queue>         // queue, priority_queue
#include <stack>         // stack
#include <map>           // map
#include <unordered_map> // unordered_map
#include <set>           // set
#include <unordered_set> // unordered_set
#include <algorithm>    // sort, reverse, etc.
#include <numeric>      // accumulate
#include <climits>      // INT_MAX, INT_MIN
#include <cmath>        // math functions
#include <cstring>      // memset, memcpy

```

7.2 常见陷阱

```

// 1. cin >> 后使用 getline 的问题
cin >> n;
cin.ignore(); // 必须忽略换行符
getline(cin, line);

// 2. 整数溢出
long long sum = 0; // 使用 long long 避免溢出
for (int num : nums) {
    sum += num;
}

// 3. 浮点数比较
const double EPS = 1e-9;
if (fabs(a - b) < EPS) {
    // a 和 b 相等
}

// 4. 数组越界
vector<int> nums(n);
// 访问 nums[n] 会越界！

// 5. 忘记初始化
vector<int> dp(n + 1, 0); // 正确
vector<int> dp(n + 1);    // 默认为0，但最好显式初始化

```


7.3 性能优化技巧

```
// 1. IO加速
ios::sync_with_stdio(false);
cin.tie(nullptr);

// 2. 预分配空间
vector<int> nums;
nums.reserve(100000); // 预分配，避免多次扩容

// 3. 使用引用传递
void func(const vector<int>& nums) {
    // 避免拷贝
}

// 4. 使用emplace_back代替push_back
vector<pair<int, int>> pairs;
pairs.emplace_back(1, 2); // 原地构造，更高效

// 5. 避免不必要的拷贝
for (const auto& item : container) {
    // 使用引用
}
```